



프로그램 언어

12주차



인하대학교

Lab: 매출 파일 처리

- 입력 파일에 상점의 일별 매출이 저장되어 있다고 하자. 이것을 읽어서 일별 평균 매출과 총 매출을 계산한 후에 다른 파일에 출력하는 프로그램을 작성해보자.

sales.txt

```
1000000  
1000000  
1000000  
500000  
1500000
```

summary.txt

```
총매출 = 5000000  
평균 일매출 = 1000000.0
```



Sol: 매출 파일 처리

```
infile = open('sales.txt', "r")
outfile = open('summary.txt', "w")

sum = 0
count = 0

while True:
    line = infile.readline()
    if line == "" :
        break
    s = int(line)
    sum += s
    count += 1

outfile.write("총매출 = "+ str(sum)+"\n")
outfile.write("평균 일매출 = "+ str(sum/count) )

infile.close()
outfile.close()
```



예외처리

예외처리

- 사용자들은 잘못된 데이터를 입력할 수도 있고, 우리가 오픈하고자 하는 파일이 컴퓨터에 존재하지 않을 수도 있으며 인터넷이 다운될 수도 있다.

```
>>> (x, y)=(2, 0)
```

```
>>> z=x/y
```

```
Traceback (most recent call last):
```

```
  File "<pyshell#1>", line 1, in <module>
```

```
    z=x/y
```

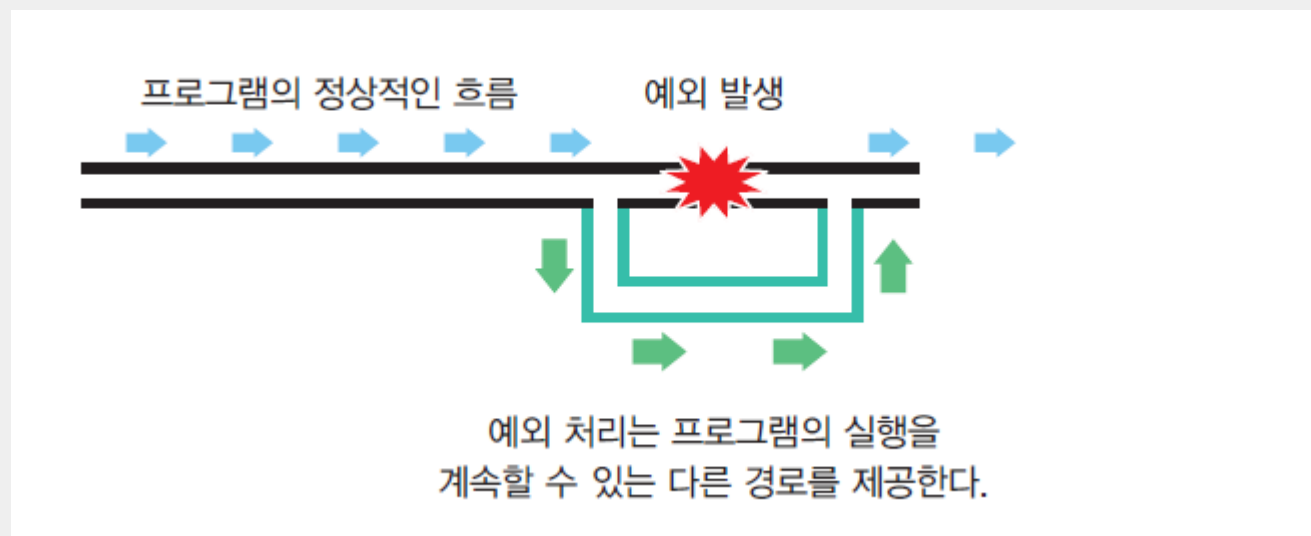
```
ZeroDivisionError: division by zero
```

```
>>>
```



예외처리

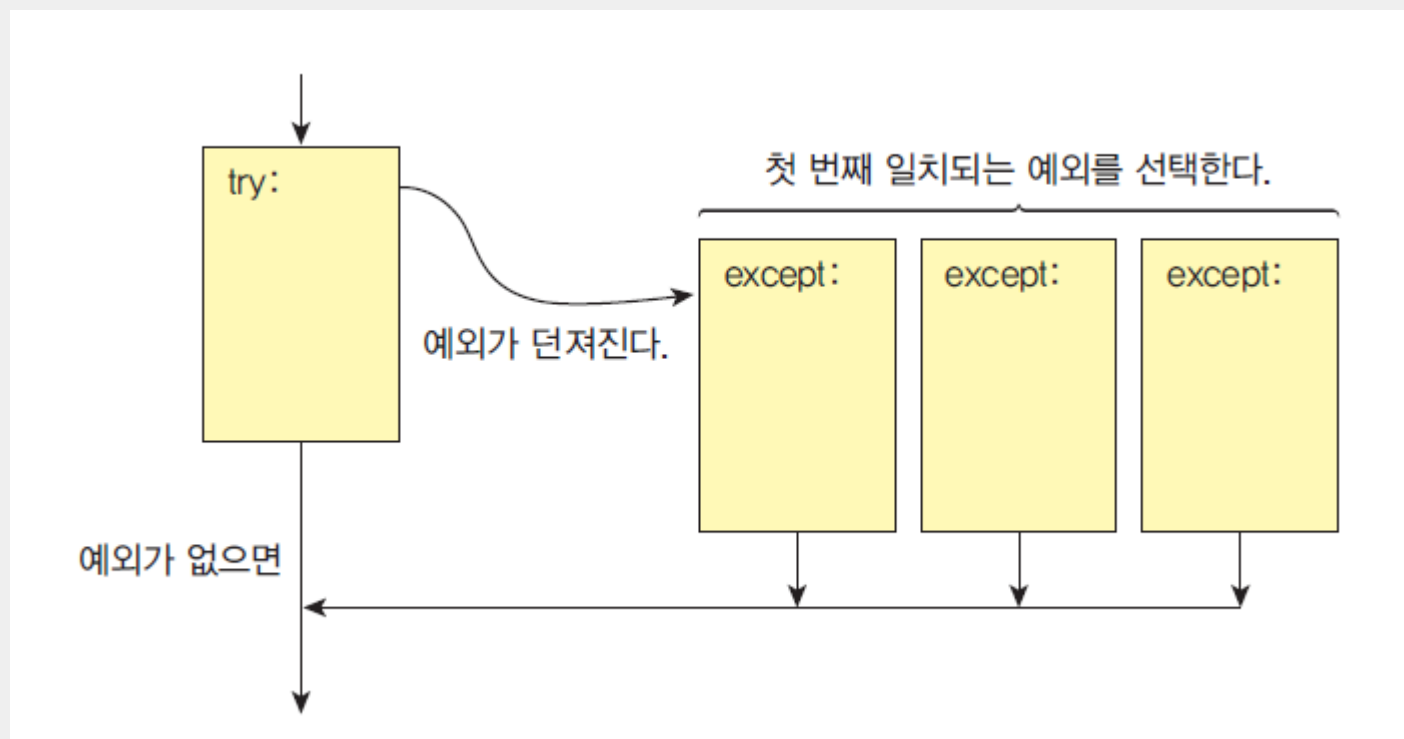
- 오류가 발생했을 때 오류를 사용자에게 알려주고 모든 데이터를 저장하게 한 후에 사용자가 **우아하게(gracefully)** 프로그램을 종료할 수 있도록 하는 것이 바람직



오류의 종류

- **사용자 입력 오류**: 사용자가 정수를 입력하여야 하는데 실수를 입력할 수 있다.
- **장치 오류**: 네트워크가 안 된다거나 하드 디스크 작동이 실패할 수 있다.
- **코드 오류**: 잘못된 인덱스를 사용하여서 배열에 접근할 수 있다.
 - **IOError**: 파일을 열 수 없으면 발생한다.
 - **importError**: 파이썬이 모듈을 찾을 수 없으면 발생한다.
 - **ValueError**: 연산이나 내장 함수에서 인수가 적절치 않은 값을 가지고 있으면 발생한다.
 - **KeyboardInterrupt**: 사용자가 인터럽트 키를 누르면 발생한다. (Ctrl-C나 Delete)
 - **EOFError**: 내장 함수가 파일의 끝을 만나면 발생한다.

Try-except 구조



Try-except 구조

try 블록에서 발생한 예외를
except 블록에서 처리

Syntax: 예외 처리

형식 try:
 예외가 발생할 수 있는 문장
except(오류):
 예외를 처리하는 문장

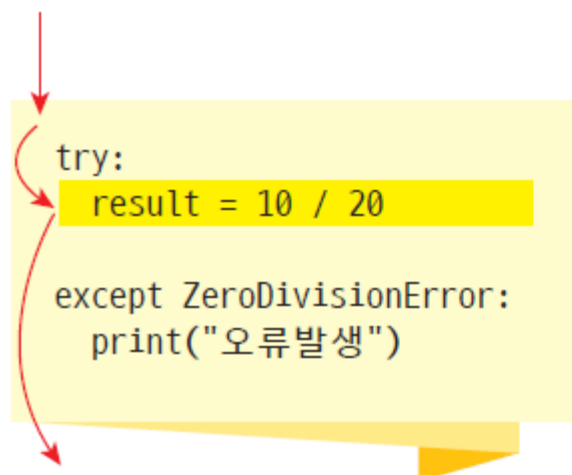
except 에 예외를 적지 않으면
어떠한 예외라도 처리

예 try:
 z = x/y
except ZeroDivisionError:
 print ("0으로 나누는 예외")

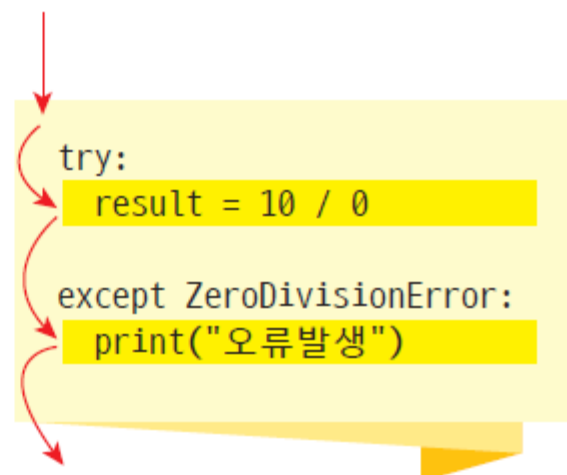
예외가 발생할 수 있는 문장

예외

try/except 블록에서의 실행 흐름



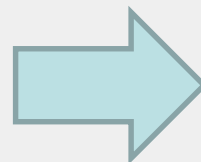
예외가 발생하지 않은 경우



예외가 발생 한 경우

예제

```
a=int(input("first number :"))  
b= int(input("second number :"))  
x=a/b  
print("나눈 결과는",x,"입니다")
```



```
a=int(input("first number :"))  
b= int(input("second number :"))  
try:  
    x=a/b  
    print("나눈 결과는",x,"입니다")  
except ZeroDivisionError:  
    print("0으로 나누면 안됩니다. ")
```

```
first number : 10  
second number : 0  
Traceback (most recent call last):  
  File "C:/Users/INHA/Desktop/test.py", line 3, in <module>  
    x=a/b  
ZeroDivisionError: division by zero
```

예제

```
(x,y) = (2,0)
try:
    z = x/y
except ZeroDivisionError:
    print("0으로 나누는 예외")
```

0으로 나누는 예외

```
(x,y) = (2,0)
try:
    z = x/y
except ZeroDivisionError as e:    #예외메세지 출력
    print(e)
```

division by zero

예시

```
>>> n=int(input('숫자를 입력하시오'))
숫자를 입력하시오 12.9
Traceback (most recent call last):
  File "<pyshell#1>", line 1, in <module>
    n=int(input('숫자를 입력하시오'))
ValueError: invalid literal for int() with base 10: '12.9'
```

```
try:
    n = input("숫자를 입력하시오 : ")
    n = int(n)
except ValueError:
    print("정수가 아닙니다. ")

print("정수 입력 성공!")
```

정수입력 여부 판단

```
while True:
    try:
        n = input("숫자를 입력하시오 : ")
        n = int(n)
        break
    except ValueError:
        print("정수가 아닙니다. 다시 입력하시오. ")
print("정수 입력 성공!")
```

```
숫자를 입력하시오 : 23.5
정수가 아닙니다. 다시 입력하시오.
숫자를 입력하시오 : 10
정수 입력 성공!
```

예제

```
try:  
    fname = input("파일 이름을 입력하세요: ")  
    infile = open(fname, "r")  
except IOError:  
    print("파일 " + fname + "을 발견할 수 없습니다.")
```

```
파일 이름을 입력하세요: kkk.py  
파일 kkk.py을 발견할 수 없습니다.
```

다중 예외 처리 구조

try:

```
fh = open("testfile", "w")
```

```
fh.write("테스트 데이터를 파일에 씁니다!!")
```

except IOError:

```
print("Error: 파일을 찾을 수 없거나 데이터를 쓸 수 없습니다. ")
```

else: #try 블록이 아무런 예외를 발생하지 않을 때

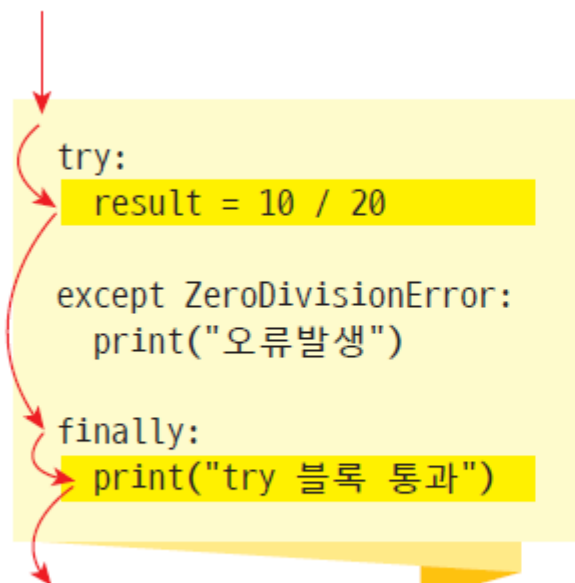
```
print("파일에 성공적으로 기록하였습니다. ")
```

```
fh.close()
```

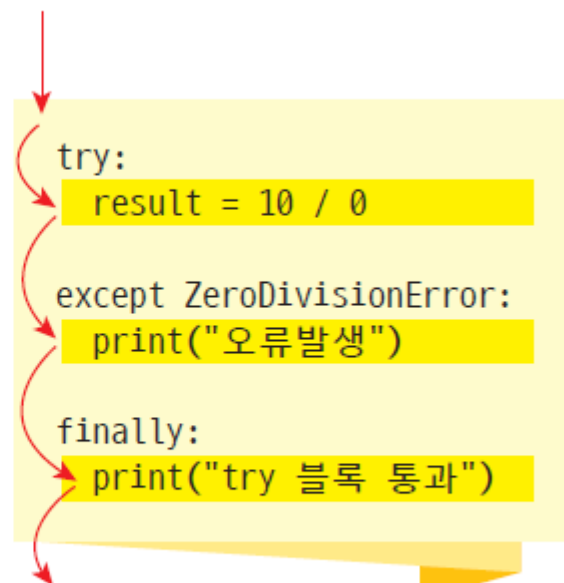
파일에 성공적으로 기록하였습니다.

finally 블록

: 오류가 발생한 경우나 발생하지 않은 경우에도 항상 실행



예외가 발생하지 않은 경우



예외가 발생하는 경우

finally 블록의 사용 예

```
try:
    f = open("test.txt", "w" )
    f.write("테스트 데이터를 파일에 씁니다!!")
    ...                               # 파일 연산을 수행한다.
except IOError:
    print("Error: 파일을 찾을 수 없거나 데이터를 쓸 수 없습니다. ")
finally:
    f.close()
```



인하대학교



라이브러리 설치방법

1. 파이참 : File-Setting-Python Interpreter에서 + 클릭 후 **numpy** 찾아서

Install Package

2. 시작메뉴 cmd에서 **pip install numpy**

3. **pandas, matplotlib** 까지 설치



Pip 오류

- Cmd 관리자모드
- Pip 실행파일이 있는 Scripts폴더위치 확인

- 커맨드라인에서

```
cd C:\Users\WINHA\AppData\Local\Programs\Python\Python311\Scripts
```

- pip install 패키지명

데이터처리 패키지

1. 데이터 수집 : 공공데이터
2. 데이터 처리 : 넘파이(numpy), 판다스(pandas)
3. 데이터 가시화 : Matplotlib



공공데이터

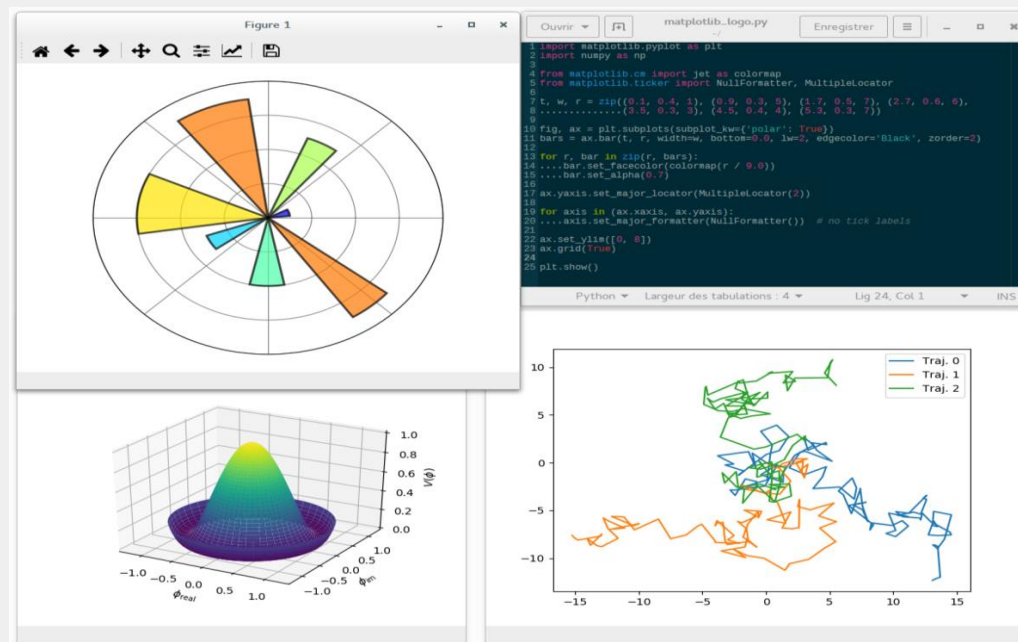
- 공공데이터 포털 : data.go.kr
- 기상자료 : data.kma.go.kr
- 행안부포털(인구통계) : mois.go.kr
- 교통카드 포털 : pay.tmoney.co.kr



메트플롯(Matplotlib)

MatPlot

- MatPlot은 그래프를 그리는 라이브러리이다.
- 점이나 막대를 이용하여 데이터의 상관관계를 알려주는 2차원 또는 3차원 그림
- Matplotlib의 하위모듈인 pyplot 사용([matplotlib.pyplot](#))



직선 그래프

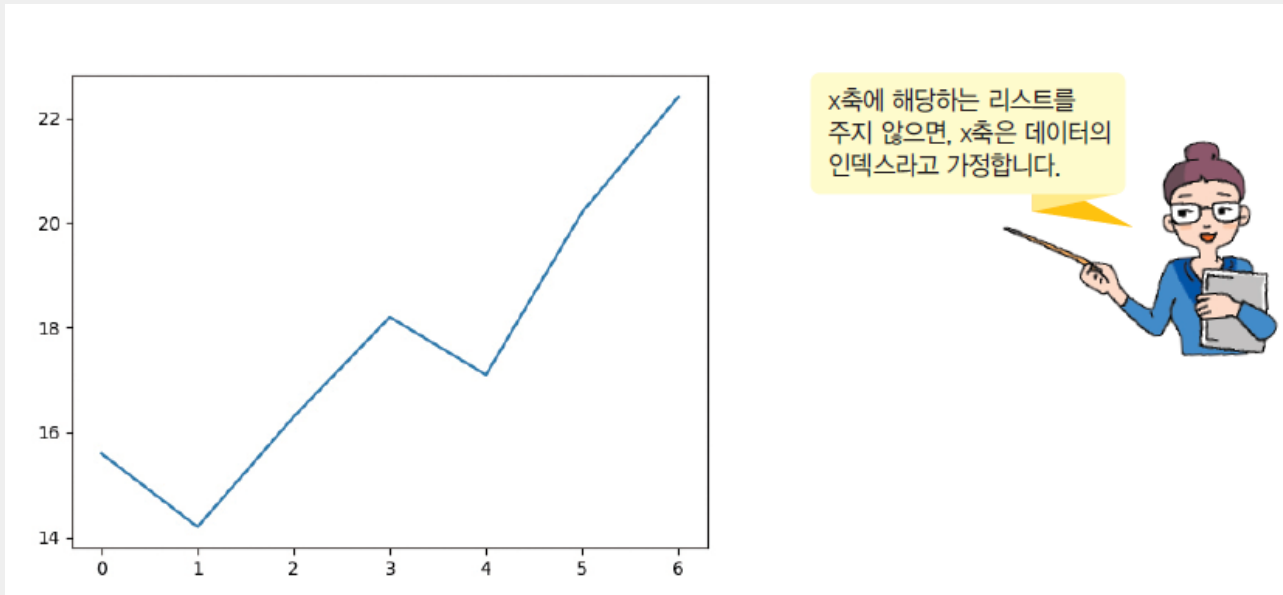


인하대학교

`plt.plot(x,y)`

```
import matplotlib.pyplot as plt
```

```
plt.plot([15.6, 14.2, 16.3, 18.2, 17.1, 20.2, 22.4])  
plt.show()
```



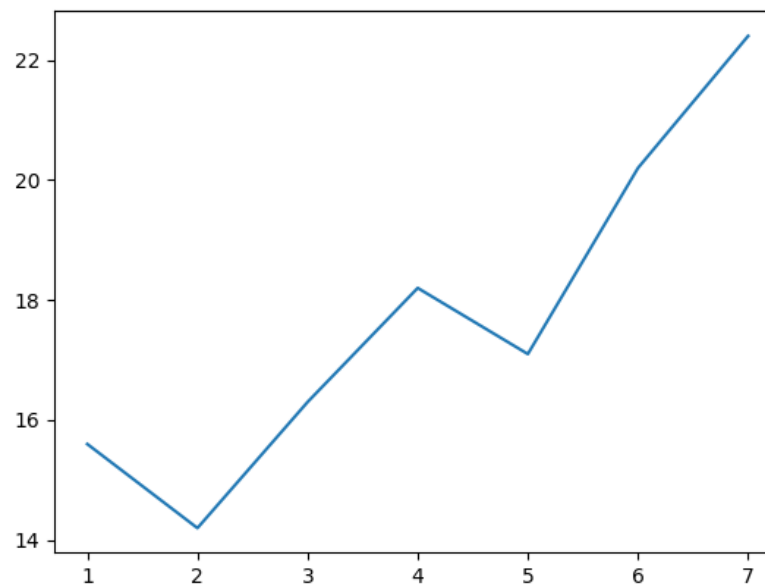
직선 그래프

```
X = [ 1, 2, 3, 4, 5, 6, 7]
```

```
Y = [15.6, 14.2, 16.3, 18.2, 17.1, 20.2, 22.4]
```

```
plt.plot(X, Y)
```

```
plt.show()
```



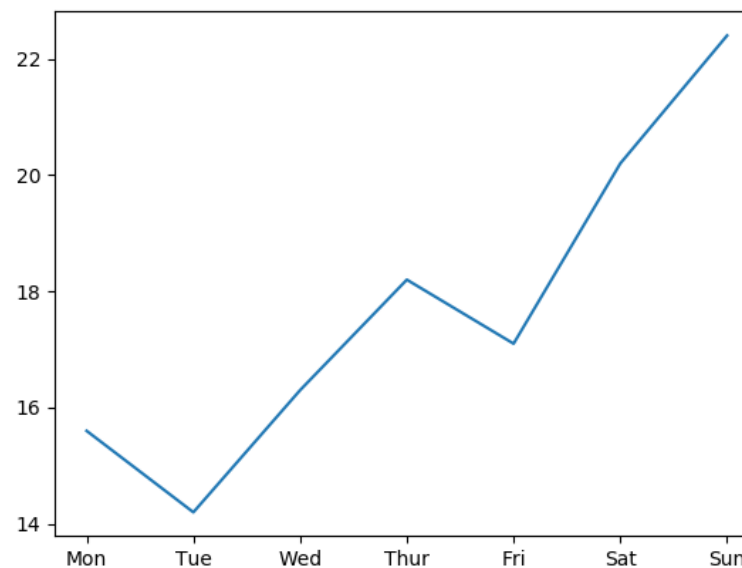
직선 그래프

```
X = [ "Mon", "Tue", "Wed", "Thur", "Fri", "Sat", "Sun" ]
```

```
Y = [15.6, 14.2, 16.3, 18.2, 17.1, 20.2, 22.4]
```

```
plt.plot(X, Y)
```

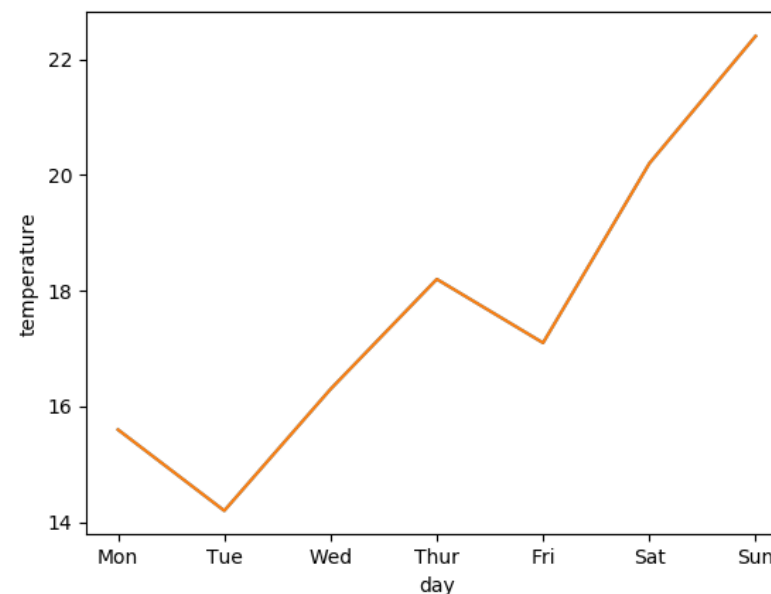
```
plt.show()
```



직선 그래프

```
X = [ "Mon", "Tue", "Wed", "Thur", "Fri", "Sat", "Sun" ]  
Y = [ 15.6, 14.2, 16.3, 18.2, 17.1, 20.2, 22.4 ]
```

```
plt.plot(X, Y)  
plt.xlabel("day")  
plt.ylabel("temperature")  
plt.show()
```



직선 그래프

```
X = [ "Mon", "Tue", "Wed", "Thur", "Fri", "Sat", "Sun" ]
```

```
Y1 = [15.6, 14.2, 16.3, 18.2, 17.1, 20.2, 22.4]
```

```
Y2 = [20.1, 23.1, 23.8, 25.9, 23.4, 25.1, 26.3]
```

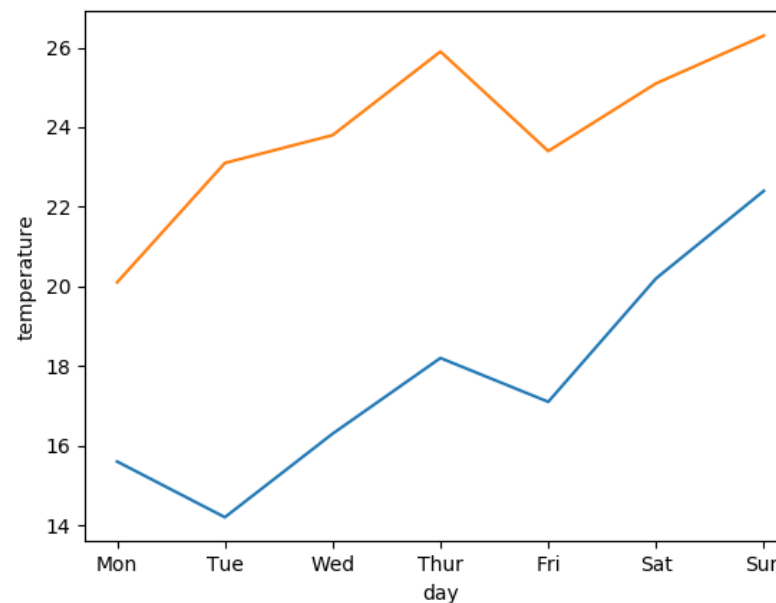
```
plt.plot(X, Y1, X, Y2)
```

```
plt.xlabel("day")
```

```
plt.ylabel("temperature")
```

```
plt.show()
```

plot()에 2개의 쌍을 보낸다.



직선 그래프

```
X = [ "Mon", "Tue", "Wed", "Thur", "Fri", "Sat", "Sun" ]
```

```
Y1 = [15.6, 14.2, 16.3, 18.2, 17.1, 20.2, 22.4]
```

```
Y2 = [20.1, 23.1, 23.8, 25.9, 23.4, 25.1, 26.3]
```

```
plt.plot(X, Y1, label="Seoul")
```

분리시켜서 그려도 됨

```
plt.plot(X, Y2, label="Busan")
```

분리시켜서 그려도 됨

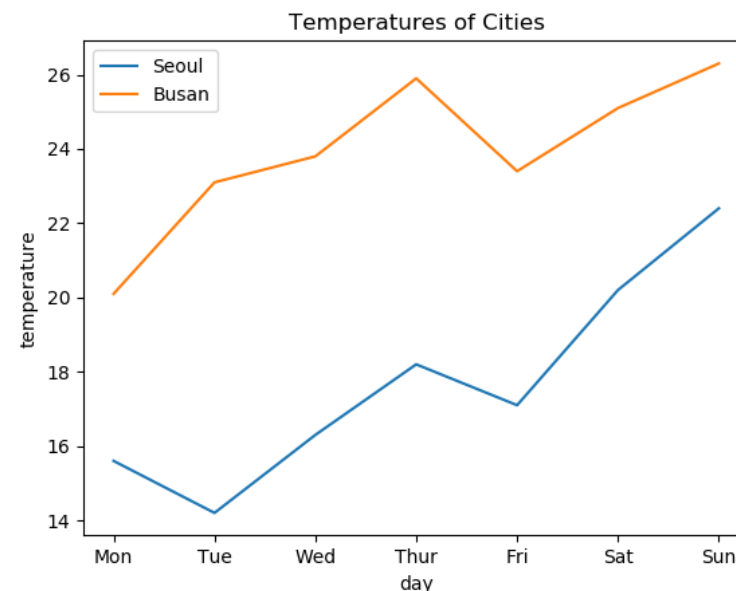
```
plt.xlabel("day")
```

```
plt.ylabel("temperature")
```

```
plt.legend(loc="upper left")
```

```
plt.title("Temperatures of Cities")
```

```
plt.show()
```



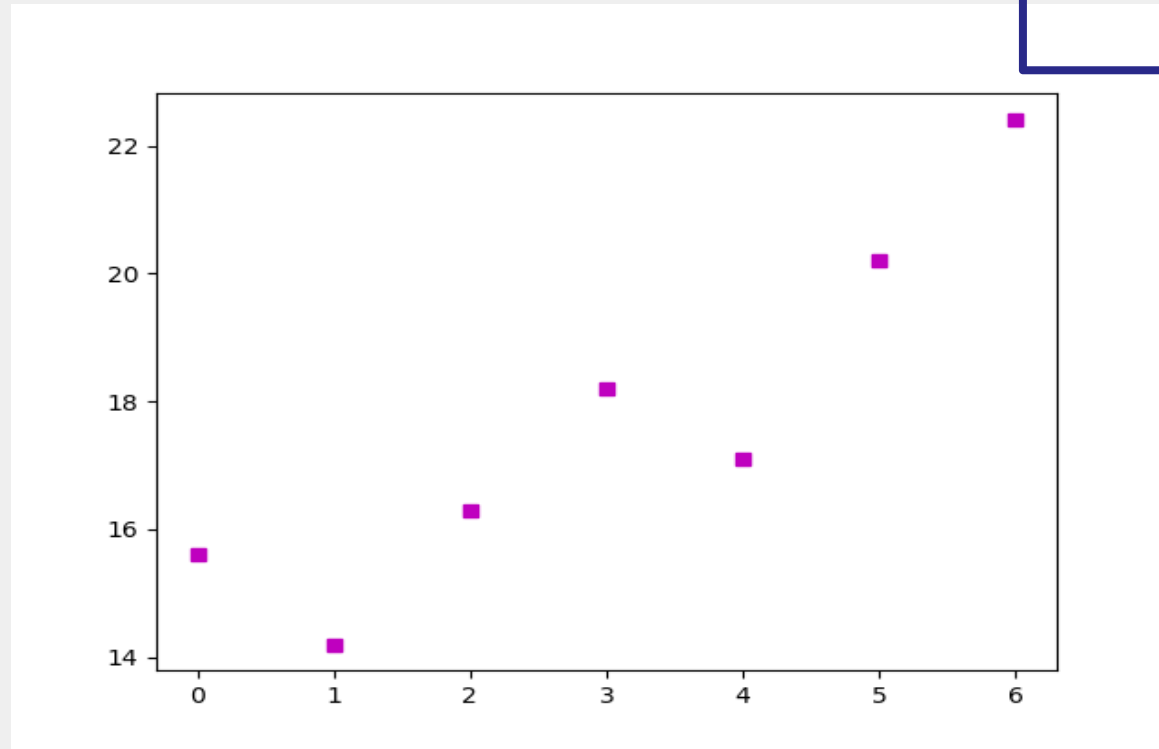
점선 그래프



인하대학교

s : square marker
m: magenta

```
plt.plot([15.6, 14.2, 16.3, 18.2, 17.1, 20.2, 22.4], "sm")  
plt.show()
```



마커의 종류

“_”
“*”
“o”
“.”
“D”
“v”

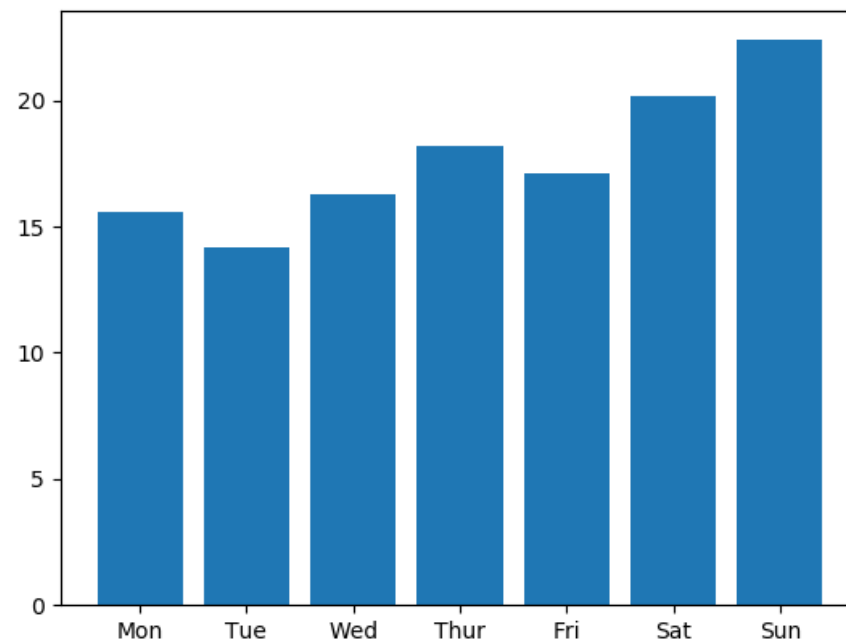
막대 그래프

```
X = [ "Mon", "Tue", "Wed", "Thur", "Fri", "Sat", "Sun" ]
```

```
Y = [15.6, 14.2, 16.3, 18.2, 17.1, 20.2, 22.4]
```

```
plt.bar(X, Y)
```

```
plt.show()
```





넘파이(numpy)

넘파이의 기초

- 넘파이(NumPy, Numerical Python)는 행렬(matrix) 계산을 위한 파이썬 라이브러리 모듈이다.
- 처리 속도가 중요한 인공지능이나 데이터 과학에서는 파이썬의 리스트 대신에 넘파이를 선호한다.
- 기계학습프로그램 작성에 사용되는 scikit-learn이나 tensorflow 패키지도 모두 넘파이 위에서 작동

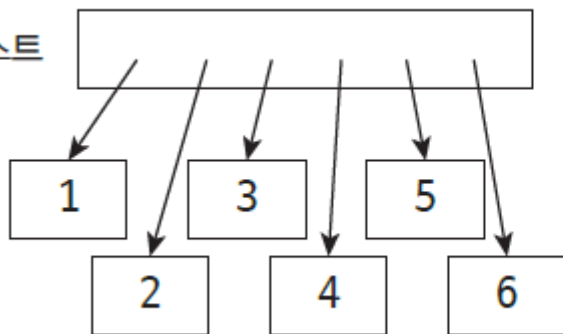


파이썬의 리스트(list) vs 넘파이

넘파이 배열

1 2 3 4 5 6

리스트



리스트에서는 데이터가
흩어져 있어서 다음
데이터를 찾기 어렵네요,



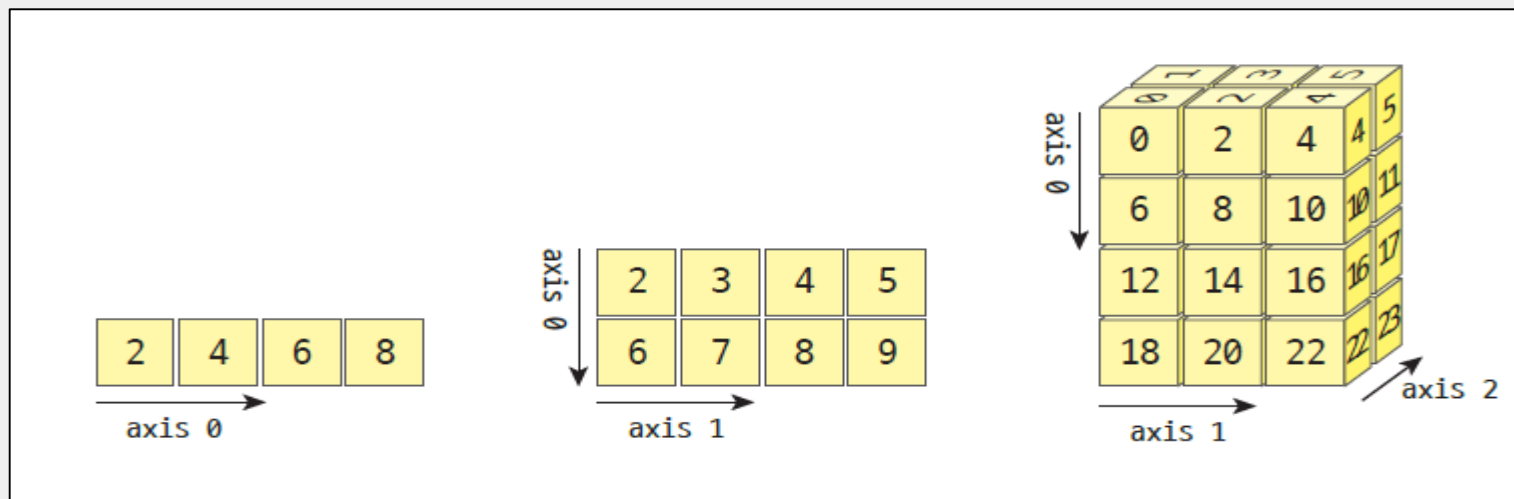
데이터가 연속적인
공간에 있어야 다음
데이터를 찾기가
쉽습니다.



```
>>> a=[1,2, '인하대']
>>> id(a[0])
2436600824048
>>> id(a[1])
2436600824080
>>> id(a[2])
2436636123344
```

넘파이 배열의 종류

- 넘파이에서는 ndarray(n차원 배열) 사용
- 동일한 자료형의 항목을 저장(혼합할 경우 문자형으로 전환)
- `import numpy as np`



넘파이 배열 생성

```
>>> import numpy as np
```

```
# 우리가 하고 하자.
```

```
>>> ftemp
```

```
# 넘파이 배열
```

```
>>> F = np
```

```
>>> F
```

```
array([63, 73, 80, 86, 84, 78, 66, 54, 45, 63])
```

a = np.array([1, 2, 3])

배열 객체

생성자 함수

파이썬 리스트

a = np.array([1, 2, 3])

배열 객체

생성자 함수

파이썬 리스트

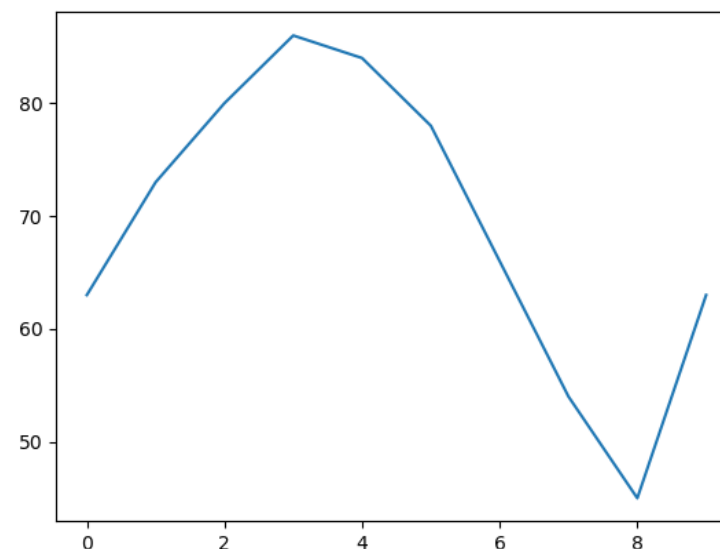
고 하자.

호출한다.

화씨온도를 섭씨온도로 바꾸고 싶으면

```
>>> F
array([63, 73, 80, 86, 84, 78, 66, 54, 45, 63])
>>> (F-32)*5/9          # 배열의 모든 요소에 이 연산이 적용된다.
array([17.22222222, 22.77777778, 26.66666667, 30.        , 28.88888889,
       25.55555556, 18.88888889, 12.22222222,  7.22222222, 17.22222222])
```

```
>>> import matplotlib.pyplot as plt
>>> plt.plot(F)
>>> plt.show()
```



배열 간 연산

```
>>> A = np.array([1, 2, 3, 4])
```

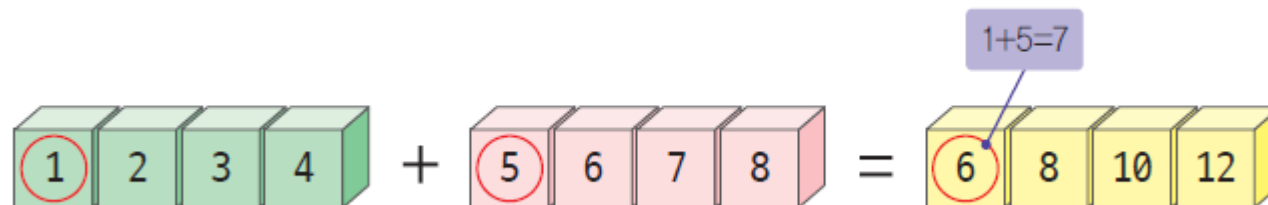
```
>>> B = np.array([5, 6, 7, 8])
```

```
>>> result = A + B
```

넘파이 배열에 + 연산이 적용된다.

```
>>> result
```

```
array([ 6,  8, 10, 12])
```



배열 간 연산

- $+$, $-$, $*$, $/$ 등의 연산자 사용
- `add()`, `subtract()`, `multiply()`, `divide()` 등의 함수로 사용가능

```
>>> A = np.array([1, 2, 3, 4])
>>> B = np.array([5, 6, 7, 8])
>>> result = A * B           # result=np.multiply(A,B)
>>> result
array([ 5, 12, 21, 32])
```

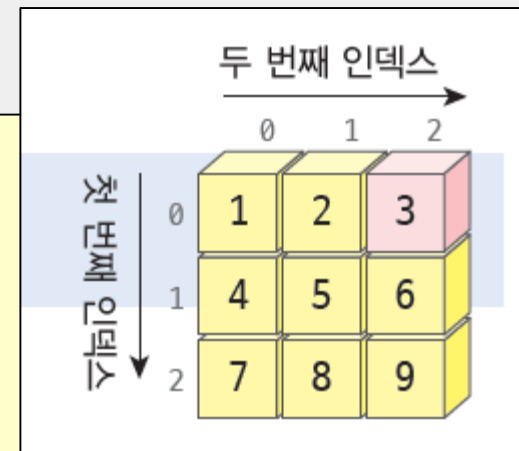
모든 연산자 가능

```
>>> a = np.array([1,2,3])
>>> b = np.array([10,20,30])
>>> 2*a
array([2,4,6])
>>> a==2
array([False, True, False])
>>> (a==2) & (b>10)
array([False, True, False])
```

2차원 배열

```
>>> b = np.array([[1, 2, 3], [4, 5, 6], [7, 8, 9]])  
>>> b  
array([[1, 2, 3],  
       [4, 5, 6],  
       [7, 8, 9]])
```

```
>>> b[0][2]  
3
```



배열객체의 속성

- **ndim** : 배열의 차원
- **size** : 전체요소의 개수
- **dtype** : 배열에 저장된 자료형
- **shape** : 각 차원의 크기

예. 2행 3열인 2차원 배열 s가 있다면

s.shape : 2,3

s.ndim : 2

s.size : 6

실습: BMI 계산하기

- 병원에서 실험 대상자들의 체질량 지수(BMI: Body Mass Index)를 계산하고자 한다.

```
heights = [ 1.83, 1.76, 1.69, 1.86, 1.77, 1.73 ]  
weights = [ 86,   74,   59,   95,   80,   68 ]
```

$$BMI = \frac{Weight(kg)}{[Height(m)]^2}$$

코딩: BMI 계산하기

```
import numpy as np
```

```
heights = [ 1.83, 1.76, 1.69, 1.86, 1.77, 1.73 ]
```

```
weights = [ 86, 74, 59, 95, 80, 68 ]
```

```
np_heights = np.array(heights)
```

```
np_weights = np.array(weights)
```

```
bmi = np_weights/(np_heights**2)
```

```
print(bmi)
```

```
[25.68007405 23.88946281 20.65754 27.45982194 25.53544639  
22.72043837]
```



넘파이 배열 생성방법

- 파이썬 리스트를 이용하는 방법
- Numpy에서 제공하는 함수를 사용하는 방법

넘파이 함수를 이용한 배열생성

- `zeros(행,열)` : 해당배열에 모두 0을 채워 생성 예.`np.zeros((2,2))`
- `ones(행,열)` : 해당배열에 모두 1을 채워 생성 예.`np.ones((2,3))`
- `full(행,열,값)` : 배열이 사용자가 지정한 값을 넣어 생성 예.`np.full((2,3),7)`
- `eye(행)` : 대각선은1, 나머지는 0을 갖는 행과 열이 같은 단위행렬 생성 예. `eye(3)`
- `arange(start,end,step)` : 생성된 값을 가지고 배열생성 예.`np.arange(1,21)`
- `linspace(x1,x2,n)` : x1과 x2 사이에 균일한 간격으로 n개의 값 생성(기본은 50개)
- `reshape(행,열)` : 배열을 지정한 형태로 변형하는 함수 예. `s.reshape((4,5))`

넘파이의 데이터 생성 함수: *arange()*

```
>>> A = np.arange(1, 10, 2)  
>>> A  
array([1, 3, 5, 7, 9])
```

np.arange(start, stop, step)

시작값

종료값

간격

넘파이의 데이터 생성 함수: *linspace()*

```
>>> A = np.linspace(0, 10, 100)
>>> A
array([ 0.        ,  0.1010101 ,  0.2020202 ,  0.3030303 ,  0.4040404 ,
        ...
        9.09090909,  9.19191919,  9.29292929,  9.39393939,  9.49494949,
        9.5959596 ,  9.6969697 ,  9.7979798 ,  9.8989899 , 10.        ])
```

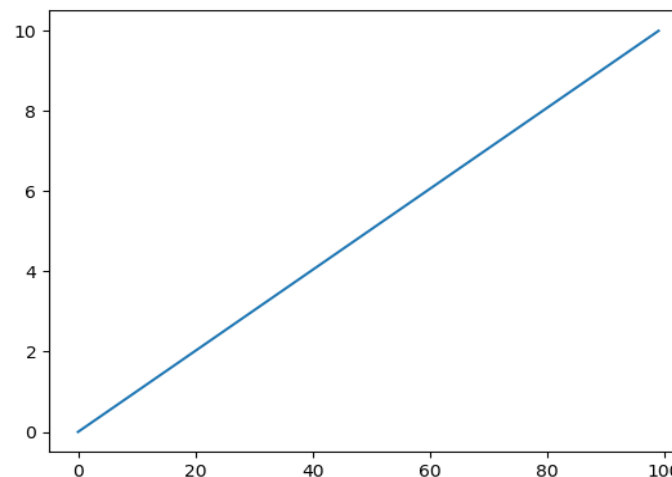
```
>>> import matplotlib.pyplot as plt
>>> plt.plot(A)
>>> plt.show()
```

`np.linspace(start, stop, num)`

시작값

종료값

개수



넘파이 내장함수

- **sum(배열, axis=n)** : 각 배열의 요소를 더함. axis=1 행끼리, axis=0 열끼리 더하기

```
a=np.array([[1,2],[3,4]])
```

```
np.sum(a)    # a.sum()
```

```
np.sum(a, axis=0)
```

- **prod(배열, axis=n)** : 각 배열의 요소 곱하기, axis=1 행끼리, axis=0 열끼리 곱하기

```
np.prod(a)
```

내장함수

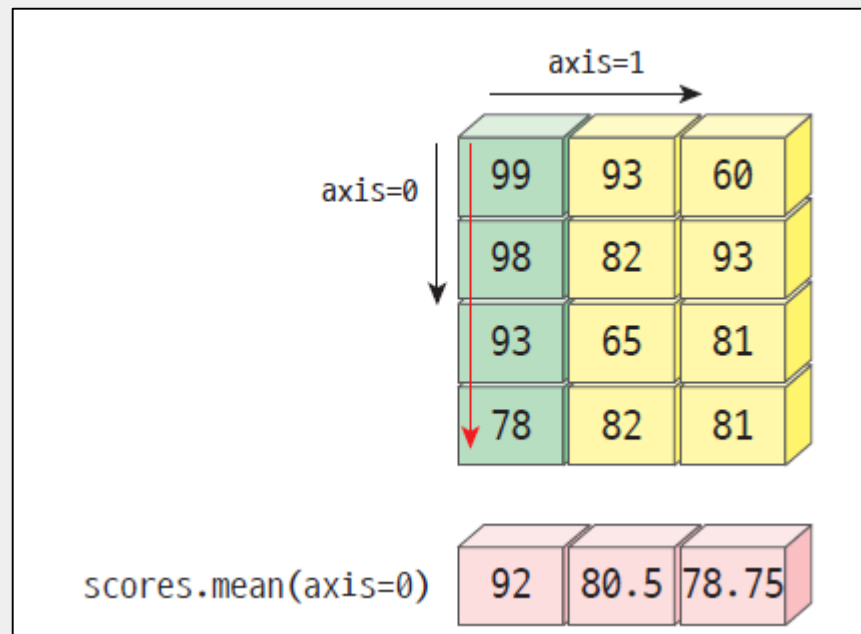
- 학생 4명의 3과목 성적(국어, 영어, 수학)이 넘파이 배열에 저장되었다고 가정하자.

```
>>> import numpy as np
>>> scores = np.array([[99, 93, 60], [98, 82, 93],
...:                   [93, 65, 81], [78, 82, 81]])

>>> scores.sum()    #np.sum(scores)와 동일, axis 사용가능
1005
>>> scores.min()
60
>>> scores.max()
99
>>> scores.mean()
83.75
>>> scores.std()    #표준편차
11.769487386175038
>>> scores.var()    #분산
138.52083333333334
```

행이나 열 단위로 계산 가능

```
>>> scores.mean(axis=0)  
array([92. , 80.5 , 78.75])
```





인덱싱과 슬라이싱

```
>>> grades = np.array([ 88, 72, 93, 94])
```

```
>>> grades[1:3]  
array([72, 93])
```

```
>>> grades[:2]  
array([88, 72])
```

2차원 배열의 슬라이싱

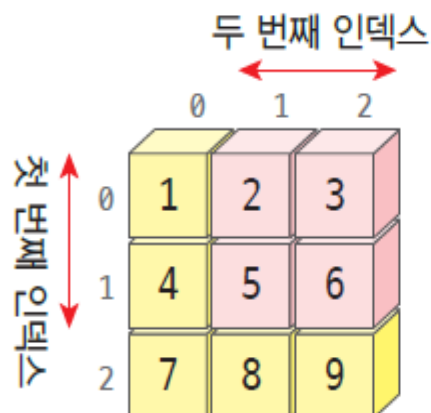
```
>>> a = np.array([[1, 2, 3], [4, 5, 6], [7, 8, 9]])
```

```
>>> a[0:2, 1:3]
```

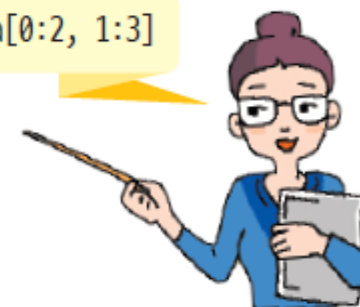
```
array([[2, 3],  
       [5, 6]])
```

```
>>> a[1:, 0:]
```

```
array([[4, 5, 6],  
       [7, 8, 9]])
```



a[0:2, 1:3]



논리적인 인덱싱(Boolean Indexing)

- 배열 각 요소의 선택여부를 True, False로 표현하는 방식.
- 불리안 인덱스를 사용하여 True인 요소만 뽑아내는 것이 가능

```
>>> ages = np.array([18, 19, 25, 30, 28])
```

ages에서 20살 이상인 사람만 고르려고 하면

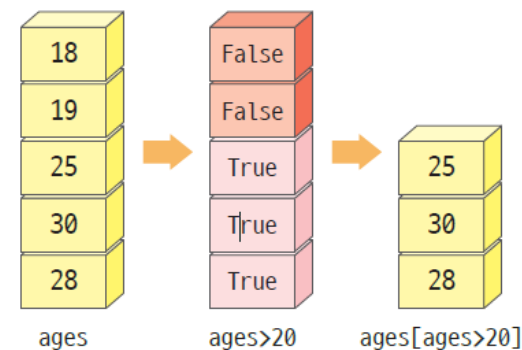
```
>>> ages > 20
```

```
array([False, False, True, True, True])
```

논리적인 인덱싱

```
>>> ages[ages > 20]
```

```
array([25, 30, 28])
```



조건을 주어서 배열 중에서 원하는 요소들을 선택할 수 있습니다.



2차원 배열의 논리적인 인덱싱

```
>>> a = np.array([[1, 2, 3], [4, 5, 6], [7, 8, 9]])
```

```
>>> a > 5
```

```
array([[False, False, False],  
       [False, False,  True],  
       [ True,  True,  True]])
```

```
>>> a[ a > 5 ]
```

```
array([6, 7, 8, 9])
```

실습1/3: 직원들의 월급 인상하기

- 현재 직원들의 월급이 [220, 250, 230]이라고 하자. 사장님이 월급을 100만원씩 올려 주기로 하셨다. 넘파이를 이용하여 계산해보자.

```
>>> import numpy as np
>>> salary = np.array([220, 250, 230])

>>> salary = salary + 100
>>> salary
array([320, 350, 330])
```



실습2/3: 직원들의 월급 인상하기

- 이것을 들은 다른 사장님은 모든 직원들의 월급을 2배 올려주기로 하셨다. 어떻게 하면 될까? 월급이 450만원 이상인 직원을 찾고 싶으면 어떻게 하면 될까?

```
>>> salary = np.array([220, 250, 230])
```

```
>>> salary = salary * 2
```

```
>>> salary
```

```
array([440, 500, 460])
```

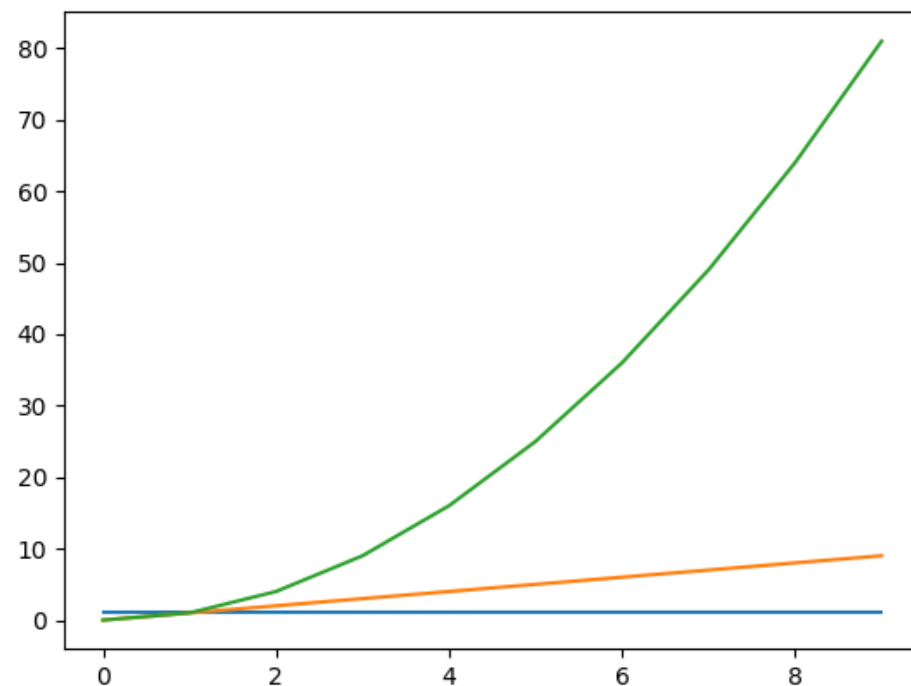
```
>>> salary > 450
```

```
array([False,  True,  True])
```



실습3/3: 그래프 그리기

- `arange(1,10)`로 x축값을 생성하고
- $f(x) = 1$, $f(x) = x$, $f(x) = x^2$ 의 그래프를 함께 그려보자.



코딩:

```
import matplotlib.pyplot as plt  
import numpy as np
```

```
X = np.arange(0, 10)
```

```
Y1 = np.ones(10)          # ones()는 1로 이루어진 넘파이 배열 생성
```

```
Y2 = X
```

```
Y3 = X**2
```

```
# 3개의 그래프를 하나의 축에 그린다.
```

```
plt.plot(X, Y1, X, Y2, X, Y3)
```

```
plt.show()
```